

Documento 03 — Documento de Desarrollo / Documento técnico

Proyecto: Sistema Inteligente de Gestión y Análisis Documental **Versión:** 1.0 **Fecha:** 8 de septiembre de 2026 **Estado:** Final

1. Introducción

Este documento describe la implementación técnica del sistema, incluyendo la estructura del código fuente, el entorno de desarrollo, las dependencias utilizadas y la integración con servicios de inteligencia artificial.

El sistema se implementa bajo la arquitectura **MERN** (MongoDB, Express, React, Node.js) con un enfoque de capas bien definidas y consumo de servicios externos de IA vía OpenRouter.

2. Estructura del código

2.1 Repositorio raíz

```
Sistema_Documental_IA/  
├─ backend/           API REST (Node.js + Express)  
├─ frontend/         Aplicación SPA (React + Vite)  
├─ 01 - Documento de Análisis/  
├─ 02 - Documento de Diseño/  
├─ 03 - Documento de Desarrollo - Documento técnico/  
├─ 04 - Plan y evidencias de Pruebas/  
├─ 05 - Documento de Implementación y Despliegue/  
├─ 06 - Manual de Usuario/  
├─ 07 - Manual Técnico - Administración/  
├─ 08 - Matriz de trazabilidad de requisitos, funcionalidades y pruebas/  
├─ 09 - Código fuente en repositorio Git/  
├─ 10 - Base de datos, scripts o estructura necesaria para reproducir el sistema/  
└─ README.md
```

2.2 Backend

```
backend/  
├─ package.json  
├─ .env.example  
├─ uploads/           Almacenamiento físico de archivos subidos
```

└─ src/	
├─ app.js	Express app, CORS, rutas, middleware de errores
├─ server.js	Punto de entrada (conexión MongoDB + listen)
├─ config/	
│ ├─ env.js	Variables de entorno (PORT, MONGODB_URI, JWT, IA)
│ └─ db.js	Conexión a MongoDB Atlas vía Mongoose
├─ models/	
│ ├─ User.js	Usuarios (name, email, password, role, company)
│ ├─ Repository.js	Repositorios (name, description, owner, members)
│ └─ Document.js	Documentos (archivo + campos IA: status, category, summary, extractedInfo, textContent, embedding)
│ └─ DocumentChunk.js	Fragmentos con embeddings para Vector Search
│ └─ ProcessingLog.js	Log de cada etapa de procesamiento
├─ controllers/	
│ ├─ authController.js	Registro, login, perfil
│ ├─ repositoryController.js	CRUD de repositorios
│ ├─ fileController.js	Upload, listado, detalle, descarga, eliminación
│ ├─ aiController.js	Disparo del procesamiento y endpoints de IA
│ └─ searchController.js	Búsqueda semántica y chat RAG
├─ routes/	
│ ├─ index.js	Agrega todos los routers bajo /api
│ ├─ authRoutes.js	/api/auth/*
│ ├─ repositoryRoutes.js	/api/repos/*
│ ├─ fileRoutes.js	/api/files/*
│ └─ searchRoutes.js	/api/search/*
├─ middleware/	
│ ├─ authMiddleware.js	Verifica JWT (protect)
│ ├─ authorize.js	Control por roles (admin, editor, lector)
│ ├─ uploadMiddleware.js	Multer con diskStorage, validación ext/MIME/tamaño
│ └─ errorMiddleware.js	Manejador centralizado de errores
├─ services/	
│ ├─ aiService.js	Cliente OpenAI-compatible (OpenRouter)
│ ├─ processingService.js	Pipeline: extracción → clasificación → resumen → extracción de campos → chunking → embeddings
│ ├─ ragService.js	Búsqueda vectorial (Atlas) + generación de respuesta
│ ├─ textExtractionService.js	Extracción de texto (PDF, DOCX, TXT)
│ └─ textChunkService.js	División en fragmentos con solape
└─ utils/	
└─ generateToken.js	Generación de JWT

2.3 Frontend

frontend/	
├─ package.json	
├─ vite.config.js	Proxy /api → Render, define VITE_API_BASE
├─ vercel.json	Reescrituras SPA + cache de assets
├─ .env.production	VITE_API_BASE=https://sistema-documental-ia.onrender.com/api
└─ src/	
├─ App.jsx	Definición de rutas (React Router v7)
├─ main.jsx	Punto de entrada
└─ index.css	Estilos globales (panel SaaS oscuro)

		api/	
			client.js Cliente HTTP (fetch con JWT), exporta api.get/post/...
		context/	
			AuthContext.jsx Estado de autenticación (user, token)
		routes/	
			ProtectedRoute.jsx Wrapper de rutas autenticadas
		components/	
			Layout.jsx Topbar + sidebar + footer
			Sidebar.jsx Navegación lateral
			RagChat.jsx Widget de chat RAG con fuente de documentos
			Footer.jsx Pie de página (créditos, GitHub)
			HelpModal.jsx Modal "Cómo usar el sistema"
			Icons.jsx Iconos SVG inline
			Alert.jsx Componente de alertas
			AuthLayout.jsx Layout para login/register
			DocumentStatus.jsx Badge de estado del documento
			Field.jsx Campo de formulario reutilizable
		pages/	
			LoginPage.jsx /login
			RegisterPage.jsx /register
			DashboardPage.jsx / – Panel principal con métricas y chat
			RepositoriesPage.jsx /repos – Lista de repositorios en tarjetas
			RepositoryDetailPage.jsx /repos/:repoId – Documentos de un repo
			DocumentsPage.jsx /documents – Listado global con filtros
			FileDetailPage.jsx /repos/:repoId/files/:docId – Detalle y análisis
			SearchPage.jsx /search – Búsqueda semántica + chat RAG

3. Entorno de desarrollo

3.1 Herramientas

Herramienta	Versión	Uso
Node.js	v24.20.0	Runtime de JavaScript
npm	11.x	Gestor de paquetes
Git	2.x	Control de versiones
VS Code	—	Editor de código
MongoDB Atlas	7.0.2	Base de datos en la nube
OpenRouter	—	Proxy de modelos de IA

3.2 Dependencias del backend (backend/package.json)

Paquete	Versión	Función
express	^4.21.0	Framework HTTP
mongoose	^8.7.0	ODM para MongoDB
bcryptjs	^2.4.3	Hash de contraseñas
jsonwebtoken	^9.0.2	Generación de tokens JWT
multer	^2.3.0	Upload de archivos (multipart)
openai	^7.10.0	Cliente compatible con OpenAI (OpenRouter)
pdfjs-dist	^6.3.289	Extracción de texto de PDFs
mammoth	^1.12.2	Extracción de texto de DOCX
dotenv	^16.6.1	Variables de entorno
cors	^2.8.5	Habilitar CORS

3.3 Dependencias del frontend (frontend/package.json)

Paquete	Versión	Función
react	^19.2.8	Biblioteca UI
react-dom	^19.2.8	Renderizado DOM
react-router-dom	^7.18.3	Enrutamiento SPA
vite	^8.2.2	Bundler y dev server
@vitejs/plugin-react	^6.1.0	Plugin de React para Vite
oxlint	^1.79.0	Linter

3.4 Scripts de ejecución

Backend:

```
cd backend
npm install          # Instalar dependencias
cp .env.example .env # Configurar variables (MONGODB_URI, JWT_SECRET, OPENAI_API_KEY)
npm run dev          # Desarrollo con --watch (auto-reinicia)
npm start            # Producción
```

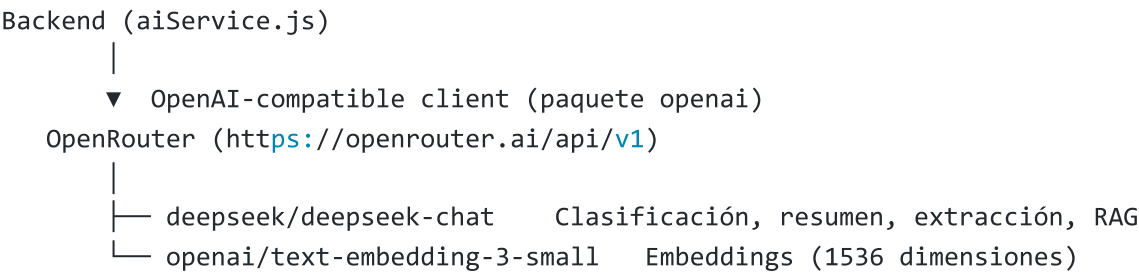
Frontend:

```
cd frontend
npm install          # Instalar dependencias
npm run dev          # Dev server en localhost:5173
npm run build        # Build de producción → dist/
npm run lint         # Linter (oxlint)
```

4. Integración con inteligencia artificial

4.1 Arquitectura de la integración

El sistema consume servicios de IA a través de **OpenRouter**, un proxy que accede a múltiples modelos de lenguaje mediante una API compatible con OpenAI. Esto permite cambiar de modelo sin modificar el código, solo ajustando variables de entorno.



4.2 Modelos utilizados

Modelo (slug)	Tarea	Configuración
deepseek/deepseek-chat	Clasificación, resumen, extracción de campos, respuestas RAG	OPENAI_MODEL
openai/text-embedding-3-small	Generación de vectores (embeddings) de 1536 dimensiones	OPENAI_EMBEDDING_MODEL

¿Por qué DeepSeek? DeepSeek-chat es un modelo de lenguaje de alto rendimiento con costo muy bajo por token. Produce respuestas en JSON estricto (compatibles con la extracción estructurada de datos) y responde bien en español, que es el idioma del sistema.

¿Por qué openai/text-embedding-3-small para embeddings? Es el modelo de embeddings más eficiente de la familia de OpenAI: 1536 dimensiones, buen balance entre calidad semántica y costo, y es directamente compatible con Atlas Vector Search (similitud coseno).

4.3 Configuración en variables de entorno

```
# Backend/.env
OPENAI_API_KEY=sk-or-v1-... # Clave de OpenRouter
OPENAI_BASE_URL=https://openrouter.ai/api/v1
OPENAI_MODEL=deepseek/deepseek-chat
OPENAI_EMBEDDING_MODEL=openai/text-embedding-3-small
```

4.4 Flujo de procesamiento de un documento

Cuando un usuario sube un documento, el backend ejecuta el pipeline de `processingService.js` de forma asíncrona:

1. `extractText(path, fileType)`
→ Extrae texto plano `del` PDF (`pdfjs-dist`), DOCX (`mammoth`) o TXT
2. `classifyDocument(text)`
→ Llama `a` DeepSeek → categoría (Contrato, Factura, Reporte, Otro)
→ Formato: `{"categoria": "Contrato"}`
3. `summarizeDocument(text)`
→ Llama `a` DeepSeek → resumen de 3-5 líneas en español
4. `extractKeyInfo(text, category)`
→ Llama `a` DeepSeek → campos clave según categoría en JSON
→ Ej. Contrato: `{partes, fecha_firma, vigencia, objeto, valor}`
5. `chunkText(text)`
→ Divide el texto en fragmentos de ~1500 caracteres con solape de 150
6. `generateEmbeddings(chunks)`
→ Llama `a` `text-embedding-3-small` → un vector de 1536 floats por chunk
7. Guarda chunks + embeddings en `DocumentChunk` (Atlas Vector Search indexable)

Si cualquier paso falla, el documento se marca como `error` con el mensaje correspondiente. El servidor **nunca se cae**: el error se registra en `ProcessingLog` y el usuario puede reintentar.

4.5 Búsqueda semántica y RAG

La búsqueda semántica utiliza **MongoDB Atlas Vector Search**:

1. Se genera el embedding de la pregunta del usuario.
2. Atlas ejecuta una consulta `$vectorSearch` sobre la colección `documentchunks` con el índice `vector_index` (similitud coseno, top-20).
3. Se recuperan los fragmentos más similares junto con los metadatos del documento origen (nombre, categoría, resumen).

4. Se arma un prompt de contexto: *"Responde con base en el siguiente contexto..."* y se envía a DeepSeek.
5. DeepSeek responde citando los documentos fuente.
6. La respuesta y las fuentes se devuelven al frontend en el widget de chat.

4.6 Resiliencia y recuperación

- **Reintentos:** el cliente de OpenAI se configura con `maxRetries: 2`.
- **Timeout:** 60 segundos por llamada a la API de IA.
- **Recovery de procesamiento colgado:** al reiniciar el servidor, `runStartupRecovery()` busca documentos en estado `processing` con más de 5 minutos sin cambio y los marca como `error` para que el usuario pueda reintentar.
- **Logs de proceso:** cada etapa se registra en `ProcessingLog` con timestamp, estado y mensaje, permitiendo diagnóstico granular.

5. Manejo de archivos

- **Upload:** Multer con `diskStorage` guarda archivos en `backend/uploads/` con nombre único `{timestamp}-{userId}-{random}.{ext}`.
- **Validación:** extensión (.pdf, .docx, .txt), MIME type y tamaño máximo (10 MB por defecto, configurable con `MAX_FILE_SIZE_MB`).
- **Descarga:** Express sirve `/uploads` como archivos estáticos; además existe el endpoint `GET /api/files/:id/download` con `res.download()`.
- **No se usa GridFS** ni almacenamiento en la nube para archivos; es filesystem local en el servidor de Render.

6. Aspectos técnicos relevantes

6.1 Autenticación

- **JWT** con expiración configurable (default 7 días).
- Token almacenado en `localStorage` del navegador.
- Middleware `protect` en todas las rutas privadas excepto `/api/auth/register`, `/api/auth/login` y `/api/health`.

6.2 Control de acceso por roles

Rol	Permisos
admin	Gestionar usuarios, eliminar cualquier documento
editor	Crear repositorios, subir/procesar documentos, eliminar los propios
lector	Buscar, consultar, preguntar (RAG), descargar

6.3 CORS

El backend permite orígenes específicos:

- `http://localhost:5173` y `http://127.0.0.1:5173` (desarrollo local)
- Cualquier `https://*.vercel.app` (producción Vercel)
- Origen definido en `FRONTEND_URL` (variable de entorno para otros hosts)

6.4 Base de datos

- **MongoDB Atlas** (cluster compartido, tier M0, versión 7.0.2).
- **Conexión:** Mongoose con la URI de `MONGODB_URI`.
- **Índice de texto** sobre `filename`, `summary`, `textContent` para búsqueda por palabra clave.
- **Índice compuesto** `{repository: 1, createdAt: -1}` para listados rápidos.
- **Índice vectorial** `vector_search` sobre `documentchunks` para búsqueda semántica (1536 dimensiones, coseno).

Fin del documento 03.